

MAD & PWA Lab

Journal

Experiment No.	07
Experiment Title.	To write meta data of your Ecommerce PWA in a Web app manifest file to enable “add to homescreen feature”.
Roll No.	02
Name	Vanshika Ambwani
Class	D15A/D15B
Subject	MAD & PWA Lab
Lab Outcome	LO4: Understand various PWA frameworks and their requirements
Grade:	

Experiment No. 7

Aim: To write meta data of your Ecommerce PWA in a Web app manifest file to enable “add to homescreen feature”.

Theory:

- **Regular Web App**

A regular web app is a website that is designed to be accessible on all mobile devices such that the content gets fit as per the device screen. It is designed using a web technology stack (HTML, CSS, JavaScript, Ruby, etc.) and operates via a browser. They offer various native-device features and functionalities. However, it entirely depends on the browser the user is using. In other words, it might be possible that you can access a native-device feature on Chrome but not on Safari or Mozilla Firefox because the browsers are incompatible with that feature.

- **Progressive Web App**

Progressive Web App (PWA) is a regular web app, but some extras enable it to deliver an excellent user experience. It is a perfect blend of desktop and mobile application experience to give both platforms to the end-users.

- **Difference between PWAs vs. Regular Web Apps:**

1. Native Experience: Though a PWA runs on web technologies (HTML, CSS, JavaScript) like a Regular web app, it gives user experience like a native mobile application. It can use most native device features, including push notifications, without relying on the browser or any other entity. It offers a seamless and integrated user experience that it is quite tough for one to differentiate between a PWA and a Native application by considering its look and feel.

2. Ease of Access: Unlike other mobile apps, PWAs do not demand longer download time and make memory space available for installing the applications. The PWAs can be shared and installed by a link, which cuts down the number of steps to install and use. These applications can easily keep an app icon on the user's home screen, making the app easily accessible to the users and helps the brands remain in the users' minds, and improving the chances of interaction.

3. Faster Services: PWAs can cache the data and serve the user with text stylesheets, images, and other web content even before the page loads completely. This lowers the waiting time for the end-users and helps the brands improve the user engagement and retention rate, which eventually adds value to their business.

4. Engaging Approach: As already shared, the PWAs can employ push notifications and other native device features more efficiently. Their interaction does not depend on the browser user uses. This eventually improves the chances of notifying the user regarding your services, offers, and other options related to your brand and keeping them hooked to your brand. In simpler words, PWAs let you maintain the user engagement and retention rate.

- 5. Updated Real:** Time Data Access: Another plus point of PWAs is that these apps get updated on their own. They do not demand the end-users to go to the App Store or other such platforms to download the update and wait until installed.
- 6. Discoverable:** PWAs reside in web browsers. This implies higher chances of optimizing them as per the Search Engine Optimization (SEO) criteria and improving the Google rankings like that in websites and other web apps.
- 7. Lower Development Cost:** Progressive web apps can be installed on the user device like a native device, but it does not demand submission on an App Store. This makes it far more cost-effective than native mobile applications while offering the same set of functionalities.

□ **The main features are:**

1. Progressive — They work for every user, regardless of the browser chosen because they are built at the base with progressive improvement principles.
2. Responsive — They adapt to the various screen sizes: desktop, mobile, tablet, or dimensions that can later become available.
3. Updated — Information is always up-to-date thanks to the data update process offered by service workers.
4. Secure — Exposed over HTTPS protocol to prevent the connection from displaying information or altering the contents.
5. Searchable — They are identified as “applications” and are indexed by search engines.
6. Reactivable — Make it easy to reactivate the application thanks to capabilities such as web notifications.
7. Installable — They allow the user to “save” the apps that he considers most useful with the corresponding icon on the screen of his mobile terminal (home screen) without having to face all the steps and problems related to the use of the app store.
8. Linkable — Easily shared via URL without complex installations.
9. Offline — Once more it is about putting the user before everything, avoiding the usual error message in case of weak or no connection. The PWA are based on two particularities: first of all the ‘skeleton’ of the app, which recalls the page structure, even if its contents do not respond and its elements include the header, the page layout, as well as an illustration that signals that the page is loading.

Code:

1. Manifest.json

```
{  
  "name": "Shopping App",  
  "short_name": "MyApp",  
  "description": "An online shopping platform with a wide range of  
products.", "start_url": "/",  
  "scope": "/",
```

```
"display": "standalone",
"background_color": "#ffffff",
"theme_color": "#ff6600",
"icons": [
  {
    "src": "assets/images/banner-1.png",
    "sizes": "192x192",
    "type": "image/png",
    "purpose": "any maskable"
  },
  {
    "src": "assets/images/banner-2.png",
    "sizes": "512x512",
    "type": "image/png",
    "purpose": "any maskable"
  }
],
"shortcuts": [
  {
    "name": "Shop Now",
    "short_name": "Shop",
    "description": "Go directly to shopping",
    "url": "/index.html",
    "icons": [
      {
        "src": "assets/images/shortcut-icon.png",
        "sizes": "96x96",
        "type": "image/png"
      }
    ]
  }
]
```

2. Service-worker.js self.addEventListener("install", (event) => {

```
console.log("Service Worker Installed");  
});
```

```
self.addEventListener("fetch", (event) => {  
event.respondWith(fetch(event.request));  
});
```

3. Script.js file:

```
'use strict';
```

```
/**  
 * add event on element  
 */
```

```
const addEventOnElem = function (elem, type, callback) {  
if (elem.length > 1) { for (let i = 0; i < elem.length; i++) {  
elem[i].addEventListener(type, callback);  
}  
} else {  
elem.addEventListener(type, callback);  
}  
}
```

```
/**  
 * navbar toggle  
 */
```

```
const navTogglers = document.querySelectorAll("[data-nav-toggler]");  
const navbar = document.querySelector("[data-navbar]"); const  
navbarLinks = document.querySelectorAll("[data-nav-link]"); const  
overlay = document.querySelector("[data-overlay]");
```

```
const toggleNavbar = function () { navbar.classList.toggle("active");  
overlay.classList.toggle("active");  
}
```

```
addEventOnElem(navTogglers, "click", toggleNavbar);
```

```
const closeNavbar = function () { navbar.classList.remove("active");  
overlay.classList.remove("active");  
}
```

```
addEventOnElem(navbarLinks, "click", closeNavbar);
```

```
/**
 * header sticky & back top btn active
 */

const header = document.querySelector("[data-header]"); const
backTopBtn = document.querySelector("[data-back-top-btn]");

const headerActive = function () {
  if (window.scrollY > 150) {
    header.classList.add("active");
    backTopBtn.classList.add("active");
  } else {
    header.classList.remove("active");
    backTopBtn.classList.remove("active");
  }
}

addEventOnElem(window, "scroll", headerActive);

let lastScrolledPos = 0;

const headerSticky = function () {  if
(lastScrolledPos >= window.scrollY) {
  header.classList.remove("header-hide");
} else {
  header.classList.add("header-hide");
}

  lastScrolledPos = window.scrollY;
}

addEventOnElem(window, "scroll", headerSticky);

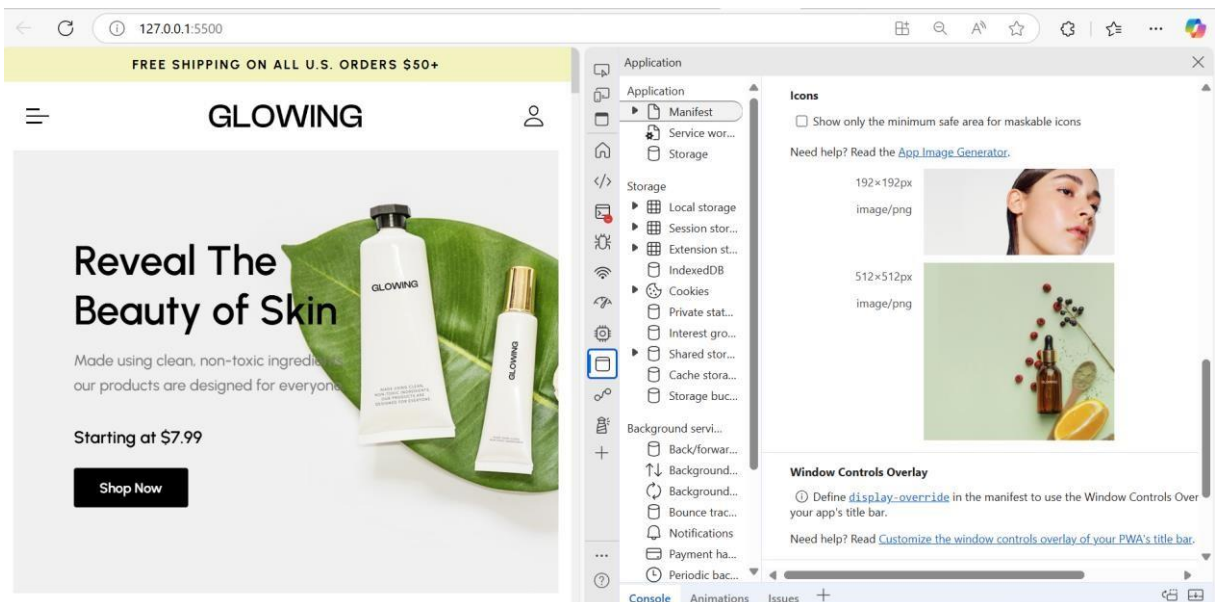
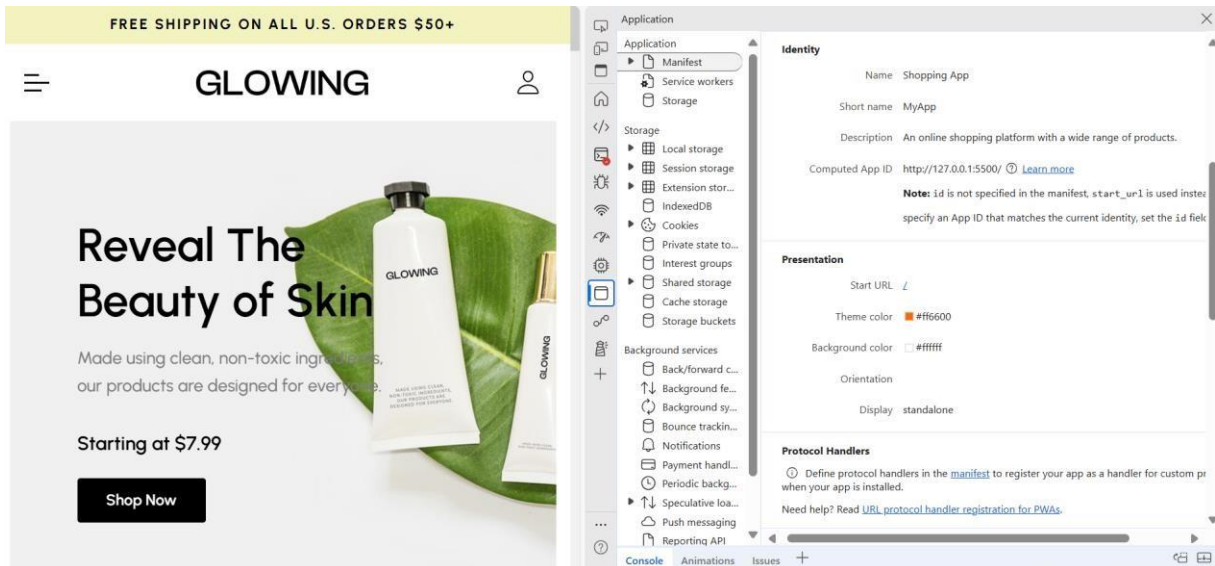
/**
 * scroll reveal effect
 */

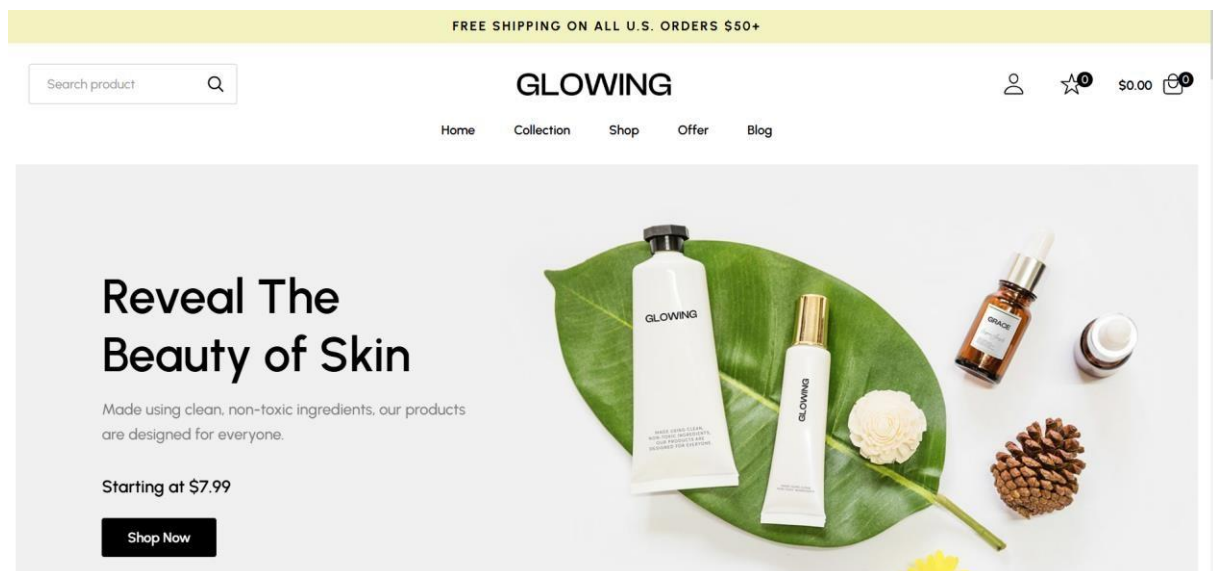
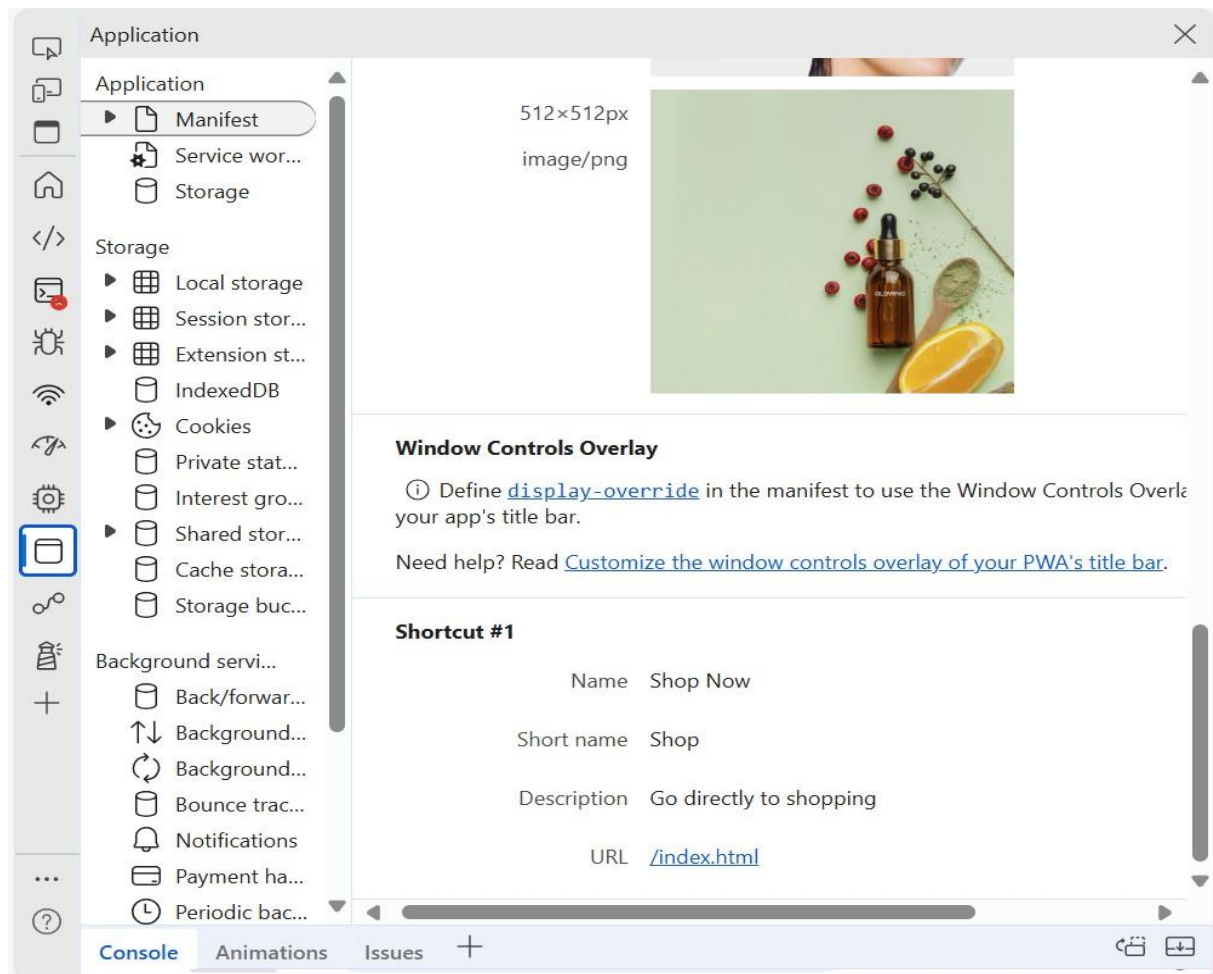
const sections = document.querySelectorAll("[data-section]");

const scrollReveal = function () {  for
(let i = 0; i < sections.length; i++) {
  if (sections[i].getBoundingClientRect().top < window.innerHeight / 2) {
    sections[i].classList.add("active");
  }
}
}

scrollReveal();

addEventOnElem(window, "scroll", scrollReveal);
Output:
```





[Shop All Products →](#)

-20%

~~\$39.00~~ **\$29.00**
Facial cleanser

★★★★★ 5170 reviews



\$29.00

Bio-shroom Rejuvenating Serum

★★★★★ 5170 reviews



\$29.00

Coffee Bean Caffeine Eye Cream

★★★★★ 5170 reviews



\$29.00

Facial cleanser

★★★★★ 5170 reviews



-20%

~~\$39.00~~ **\$29.00**

Coffee Bean Caffeine Eye Cream

★★★★★ 5170 reviews

